ASoC component callbacks

snd_soc_dai_ops

```
struct snd_soc_dai_ops {
        /*
         * DAI clocking configuration, all optional.
         * Called by soc_card drivers, normally in their hw_params.
         */
        int (*set_sysclk)(struct snd_soc_dai *dai,
                int clk_id, unsigned int freq, int dir);
        int (*set_pll)(struct snd_soc_dai *dai, int pll_id, int source,
                unsigned int freq_in, unsigned int freq_out);
        int (*set_clkdiv)(struct snd_soc_dai *dai, int div_id, int div);
        int (*set_bclk_ratio)(struct snd_soc_dai *dai, unsigned int ratio);

        /*
         * DAI format configuration
         * Called by soc_card drivers, normally in their hw_params.
```

snd_soc_dai_ops

```
 */
int (*set_fmt)(struct snd_soc_dai *dai, unsigned int fmt);
int (*xlate_tdm_slot_mask)(unsigned int slots,
        unsigned int *tx_mask, unsigned int *rx_mask);
int (*set_tdm_slot)(struct snd_soc_dai *dai,
        unsigned int tx_mask, unsigned int rx_mask,
        int slots, int slot_width);
```

snd_soc_dai_ops

```
/*
 * DAI digital mute - optional.
 * Called by soc-core to minimise any pops.
 */
int (*mute_stream)(struct snd_soc_dai *dai, int mute, int stream);

/*
 * ALSA PCM audio operations - all optional.
 * Called by soc-core during audio PCM operations.
 */
int (*startup)(struct snd_pcm_substream *,
        struct snd_soc_dai *);
void (*shutdown)(struct snd_pcm_substream *,
        struct snd_soc_dai *);
int (*hw_params)(struct snd_pcm_substream *,
```

snd_soc_dai_ops

```
                struct snd_pcm_hw_params *, struct snd_soc_dai *);
        int (*hw_free)(struct snd_pcm_substream *,
                struct snd_soc_dai *);
        int (*prepare)(struct snd_pcm_substream *,
                struct snd_soc_dai *);
[...]
};
```

hw_params

- The most useful callback
- It is used to configure the component to match the parameters of the audio stream.
- Called when the stream is ready to be played, before any data is transferred.
- If the requested parameters cannot be supported by the hardware, the `hw_params` callback can return an error code to indicate that the stream cannot be opened. Otherwise, the callback returns successfully, and the audio stream can be started.

snd_pcm_hw_params

- Holds the stream parameters
- Usually not accessed directly but through accessors:
  - params_channels : the number of channels
  - params_rate : the sample rate
  - params_width : the number of bits per sample

hw_params example

```
static int aic31xx_hw_params(struct snd_pcm_substream *substream,
                             struct snd_pcm_hw_params *params,
                             struct snd_soc_dai *dai)
{
        struct snd_soc_component *component = dai->component;
        struct aic31xx_priv *aic31xx =
snd_soc_component_get_drvdata(component);
        u8 data = 0;

        switch (params_width(params)) {
        case 16:
                break;
        case 20:
                data = (AIC31XX_WORD_LEN_20BITS <<
                        AIC31XX_IFACE1_DATALEN_SHIFT);
```

hw_params example

```
                break;
        case 24:
                data = (AIC31XX_WORD_LEN_24BITS <<
                        AIC31XX_IFACE1_DATALEN_SHIFT);
                break;
        case 32:
                data = (AIC31XX_WORD_LEN_32BITS <<
                        AIC31XX_IFACE1_DATALEN_SHIFT);
                break;
        default:
                dev_err(component->dev, "%s: Unsupported width %dn",
                        __func__, params_width(params));
                return -EINVAL;
        }
```

hw_params example

```
        snd_soc_component_update_bits(component, AIC31XX_IFACE1,
                                      AIC31XX_IFACE1_DATALEN_MASK,
                                      data);

        /*
         * If BCLK is used as PLL input, the sysclk is determined by the hw
         * params. So it must be updated here to match the input frequency.
         */
        if (aic31xx->sysclk_id == AIC31XX_PLL_CLKIN_BCLK) {
                aic31xx->sysclk = params_rate(params) *
params_width(params) *
                                  params_channels(params);
                aic31xx->p_div = 1;
        }
```

hw_params example

```
        return aic31xx_setup_pll(component, params);
}
```

aic31xx_setup_pll() then uses the parameters to set the CODEC PLLs and clocks properly. The usual ways to achieve that are to either do the calculations or prepare an array matching parameters to register values.

set_sysclk

- This sets the system clock parameters of the component, in particular which one is selected, its frequency and the direction.
- This allows the component to set up PLLs and clocks.
- This is called from the machine driver, using snd_soc_dai_set_sysclk()
- It can return an error in case the clock is not available or the frequency is not in the supported range.
- A component wide version exists, called using snd_soc_component_set_sysclk() , very rarely used.

set_sysclk example

```
static int aic31xx_set_dai_sysclk(struct snd_soc_dai *codec_dai,
                                  int clk_id, unsigned int freq, int dir)
{
        struct snd_soc_component *component = codec_dai->component;
        struct aic31xx_priv *aic31xx =
snd_soc_component_get_drvdata(component);
        int i;
[...]
        for (i = 1; i < 8; i++)
                if (freq / i <= 20000000)
                        break;
        if (freq/i > 20000000) {
                dev_err(aic31xx->dev, "%s: Too high mclk frequency %un",
                        __func__, freq);
                return -EINVAL;
        }
```

set_sysclk example

```
        aic31xx->p_div = i;

        for (i = 0; i < ARRAY_SIZE(aic31xx_divs); i++)
                if (aic31xx_divs[i].mclk_p == freq / aic31xx->p_div)
                        break;
        if (i == ARRAY_SIZE(aic31xx_divs)) {
                dev_err(aic31xx->dev, "%s: Unsupported frequency %dn",
                        __func__, freq);
                return -EINVAL;
        }

        /* set clock on MCLK, BCLK, or GPIO1 as PLL input */
        snd_soc_component_update_bits(component, AIC31XX_CLKMUX,
AIC31XX_PLL_CLKIN_MASK,
                                clk_id << AIC31XX_PLL_CLKIN_SHIFT);
```

set_sysclk example

```
        aic31xx->sysclk_id = clk_id;
        aic31xx->sysclk = freq;

        return 0;
}
```

set_fmt

- This sets the format of the PCM bus
- This is called from the machine driver, using snd_soc_dai_set_fmt()
- Available formats are:
  - SND_SOC_DAIFMT_I2S
  - SND_SOC_DAIFMT_RIGHT_J
  - SND_SOC_DAIFMT_LEFT_J
  - SND_SOC_DAIFMT_DSP_A
  - SND_SOC_DAIFMT_DSP_B
  - SND_SOC_DAIFMT_AC97
  - SND_SOC_DAIFMT_PDM

set_fmt

- Also the polarity can be changed:
  - SND_SOC_DAIFMT_NB_NF : normal bit clock + normal frame
  - SND_SOC_DAIFMT_NB_IF : normal bit clock + invert frame
  - SND_SOC_DAIFMT_IB_NF : invert bit clock + normal frame
  - SND_SOC_DAIFMT_IB_IF : invert bit clock + invert frame

set_fmt

- The clock directions can also be set:
  - SND_SOC_DAIFMT_CBP_CFP : codec clk provider and frame provider
  - SND_SOC_DAIFMT_CBC_CFP : codec clk consumer and frame provider
  - SND_SOC_DAIFMT_CBP_CFC : codec clk provider and frame consumer
  - SND_SOC_DAIFMT_CBC_CFC : codec clk consumer and frame consumer
- These used to have another name:

linux

```
/* previous definitions kept for backwards-compatibility, do not use in
new contributions */
#define SND_SOC_DAIFMT_CBM_CFM                SND_SOC_DAIFMT_CBP_CFP
#define SND_SOC_DAIFMT_CBS_CFM                SND_SOC_DAIFMT_CBC_CFP
```

set_fmt

```
#define SND_SOC_DAIFMT_CBM_CFS                    SND_SOC_DAIFMT_CBP_CFC
#define SND_SOC_DAIFMT_CBS_CFS                    SND_SOC_DAIFMT_CBC_CFC
```

set_fmt example

```
static int aic31xx_set_dai_fmt(struct snd_soc_dai *codec_dai,
                               unsigned int fmt)
{
        struct snd_soc_component *component = codec_dai->component;
        u8 iface_reg1 = 0;
        u8 iface_reg2 = 0;
        u8 dsp_a_val = 0;
[...]
        switch (fmt & SND_SOC_DAIFMT_CLOCK_PROVIDER_MASK) {
        case SND_SOC_DAIFMT_CBP_CFP:
                iface_reg1 |= AIC31XX_BCLK_MASTER | AIC31XX_WCLK_MASTER;
                break;
[...]
        }

        /* signal polarity */
```

set_fmt example

```
        switch (fmt & SND_SOC_DAIFMT_INV_MASK) {
        case SND_SOC_DAIFMT_NB_NF:
[...]
        }

        /* interface format */
        switch (fmt & SND_SOC_DAIFMT_FORMAT_MASK) {
[...]
        }

        snd_soc_component_update_bits(component, AIC31XX_IFACE1,
                                      AIC31XX_IFACE1_DATATYPE_MASK |
                                      AIC31XX_IFACE1_MASTER_MASK,
                                      iface_reg1);
```

set_tdm_slot

- This callback configures the DAI for TDM operation.
- `slots` is the total number of slots of the TDM stream and

```
slot_width
```

  the width of each slot in bit clock cycles.
- `tx_mask` and `rx_mask` are bitmasks specifying the active slots of the TDM stream for the specified DAI, i.e. which slots the DAI should write to or read from. A set bit means the channel is active.
- This is called from the machine driver, using snd_soc_dai_set_tdm_slot()
- This allows to explicitly configure mismatching stream and bus sample width.
- TDM mode must be disabled when `slots` is 0.

trigger

- This callback is called when the stream status is updated.
- It allows to listen for events.
- This is called from the Alsa core, in soc_pcm_trigger() using snd_soc_pcm_dai_trigger()
- A component version exists.
- Available states are:
  - SNDRV_PCM_TRIGGER_STOP
  - SNDRV_PCM_TRIGGER_START
  - SNDRV_PCM_TRIGGER_PAUSE_PUSH
  - SNDRV_PCM_TRIGGER_PAUSE_RELEASE
  - SNDRV_PCM_TRIGGER_SUSPEND

trigger

- SNDRV_PCM_TRIGGER_RESUME
- SNDRV_PCM_TRIGGER_DRAIN

`trigger` example

- The `PCM1789` needs the system clock, bit clock and frame clock to be synchronized as soon as it gets out of reset.
- With DAPM, those clocks are disabled until a stream is ready to be played.
- A solution is to reset the device when a stream is played.

trigger example

```
static int pcm1789_trigger(struct snd_pcm_substream *substream, int cmd,
                           struct snd_soc_dai *dai)
{
        struct snd_soc_component *component = dai->component;
        struct pcm1789_private *priv =
snd_soc_component_get_drvdata(component);
        int ret = 0;

        switch (cmd) {
        case SNDRV_PCM_TRIGGER_START:
        case SNDRV_PCM_TRIGGER_RESUME:
        case SNDRV_PCM_TRIGGER_PAUSE_RELEASE:
                schedule_work(&priv->work);
                break;
        case SNDRV_PCM_TRIGGER_STOP:
```

trigger example

```
        case SNDRV_PCM_TRIGGER_SUSPEND:
        case SNDRV_PCM_TRIGGER_PAUSE_PUSH:
                break;
        default:
                ret = -EINVAL;
        }

        return ret;
}
```

trigger example

linux

```
static void pcm1789_work_queue(struct work_struct *work)
{
        struct pcm1789_private *priv = container_of(work,
                                                    struct pcm1789_private,
                                                    work);

        /* Perform a software reset to remove codec from desynchronized
state */
        if (regmap_update_bits(priv->regmap, PCM1789_MUTE_CONTROL,
                               0x3 << PCM1789_MUTE_SRET, 0) < 0)
                dev_err(priv->dev, "Error while setting SRET");
}
```

set_bias_level

- This callback is called by DAPM through snd_soc_dapm_set_bias_level() and snd_soc_component_set_bias_level() once the component gets activated.
- It allows to listen for power events.
- Available events are:
  - SND_SOC_BIAS_ON : Bias is fully on for audio playback and capture operations.
  - SND_SOC_BIAS_PREPARE : Prepare for audio operations. Called before DAPM switching for stream start and stop operations.
  - SND_SOC_BIAS_STANDBY : Low power standby state when no playback/capture operations are in progress. NOTE: The transition time between STANDBY and ON should be as fast as possible and no longer than 10ms.
  - SND_SOC_BIAS_OFF : Power Off. No restrictions on transition times.

`set_bias_level` example

- There are CODECs that won't even listen on the control bus until there are clocks on the PCM bus or that will stay powered off as much as possible.
- A solution is to use regcache.

set_bias_level example

```
static int ssm2518_set_bias_level(struct snd_soc_component *component,
        enum snd_soc_bias_level level)
{
        struct ssm2518 *ssm2518 = snd_soc_component_get_drvdata(component);
        int ret = 0;

        switch (level) {
        case SND_SOC_BIAS_ON:
                break;
        case SND_SOC_BIAS_PREPARE:
                break;
        case SND_SOC_BIAS_STANDBY:
                if (snd_soc_component_get_bias_level(component) ==
SND_SOC_BIAS_OFF)
                        ret = ssm2518_set_power(ssm2518, true);
```

set_bias_level example

```
                break;
        case SND_SOC_BIAS_OFF:
                ret = ssm2518_set_power(ssm2518, false);
                break;
        }

        return ret;
}
```

set_bias_level example

```
static int ssm2518_set_power(struct ssm2518 *ssm2518, bool enable)
{
        int ret = 0;

        if (!enable) {
                ret = regmap_update_bits(ssm2518->regmap,
SSM2518_REG_POWER1,
                        SSM2518_POWER1_SPWDN, SSM2518_POWER1_SPWDN);
                regcache_mark_dirty(ssm2518->regmap);
        }

        if (ssm2518->enable_gpio)
                gpiod_set_value_cansleep(ssm2518->enable_gpio, enable);

        regcache_cache_only(ssm2518->regmap, !enable);
```

set_bias_level example

```
        if (enable) {
                ret = regmap_update_bits(ssm2518->regmap,
SSM2518_REG_POWER1,
                        SSM2518_POWER1_SPWDN | SSM2518_POWER1_RESET, 0x00);
                regcache_sync(ssm2518->regmap);
        }

        return ret;
}
```